

JavaScript + TypeScript

Interview Guide

বাংলায়, সহজ ব্যাখ্যা ও উদাহরণসহ

লক্ষ্য: শুধু মুখস্থ না করে concept বুঝে ফেলা—যাতে interview-এ প্রশ্ন যেভাবেই ঘুরিয়ে করা হোক, নিজের ভাষায়
পরিষ্কারভাবে উত্তর দিতে পারো।

70 Essential Interview Topics

Core JavaScript • Async • Performance • TypeScript • Advanced Types • Real-world Architecture

সূচিপত্র / Table of Contents

সূচিপত্র

1	PART 1: JavaScript	3
1.1	Core Language & Execution	3
1.2	Functions & Closures	4
1.3	Objects & Prototypes	6
1.4	Asynchronous JavaScript	7
1.5	Performance & Memory	9
1.6	Advanced Concepts	10
2	PART 2: TypeScript	12
2.1	Types & Interfaces	12
2.2	Functions & Generics	13
2.3	Advanced Types	14
2.4	Type Safety & Architecture	15
2.5	TS + JS Internals	17
2.6	High-Signal System / Real-World Questions	17

1 PART 1: JavaScript

1.1 Core Language & Execution

Q1. Hoisting (var, let, const, function)

JS কোড রান হওয়ার আগে একটা ``compile phase" থাকে, যেখানে variable আর function-এর ঘোষণা (declaration) memory-তে তুলে রাখা হয় --- একেই **hoisting** বলে।

- var → hoist হয় এবং undefined দিয়ে initialize হয়। তাই ব্যবহারের আগেও access করলে error না দিয়ে undefined দেখায়।
- let, const → hoist হয় কিন্তু initialize হয় না। ব্যবহারের আগে access করলে **ReferenceError** (TDZ, নিচে দেখো)।
- function declaration → পুরো function-ই hoist হয়, মানে call করার আগে define করলেও কাজ করবে।
- function expression (const দিয়ে assign করা) → শুধু ভ্যারিয়েবলটা hoist হয়, function টা না।

```
console.log(a); // undefined
var a = 5;

console.log(b); // ReferenceError
let b = 5;

greet(); // কাজ করবে
function greet() { console.log("hi"); }

sayHi(); // TypeError: sayHi is not a function
var sayHi = function() { console.log("hi"); };
```

Q2. Temporal Dead Zone (TDZ)

let/const দিয়ে declare করা variable, scope-এর শুরু থেকে declaration line পর্যন্ত যে সময়টুকু তাকে ব্যবহার করা যায় না, সেই zone-কেই TDZ বলে।

কেন আছে? যাতে ভুলে declare করার আগে variable ব্যবহার করে বাগ তৈরি না করি --- এটা একটা safety mechanism, var-এর ``silently undefined" সমস্যা এড়ানোর জন্য।

```
{
  console.log(x); // ReferenceError (TDZ)
  let x = 10;
}
```

Q3. Execution Context

যখনই কোনো JS কোড রান হয় (script load বা function call), একটা **Execution Context (EC)** তৈরি হয়। এর দুইটা phase:

1. **Creation phase** --- variable/function hoist হয়, this বাইন্ড হয়, scope chain তৈরি হয়।
2. **Execution phase** --- লাইন বাই লাইন কোড রান হয়, ভ্যালু assign হয়।

দুই ধরনের EC: **Global EC** (একটাই, পুরো script-এর জন্য) আর **Function EC** (প্রতি function call-এ নতুন একটা, call stack-এ push হয়)।

Q4. Call Stack vs Task Queue vs Microtask Queue

- **Call Stack:** synchronous কোড এখানে execute হয় --- LIFO (Last In First Out)।
- **Microtask Queue:** Promise .then/.catch/.finally, queueMicrotask, async/await-এর পরের কোড এখানে যায়।
- **Task Queue (Macrotask):** setTimeout, setInterval, DOM events, I/O --- এগুলো এখানে যায়।

Call stack খালি হলেই event loop আগে পুরো microtask queue খালি করে, তারপর একটা task queue থেকে নেয়।

Q5. Event loop-এ Promise বনাম setTimeout

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1 4 3 2
```

কারণ: call stack আগে sync কোড (1, 4) শেষ করে। তারপর event loop microtask queue (Promise → 3) সম্পূর্ণ খালি করে, সবশেষে task queue (setTimeout → 2) থেকে একটা করে নেয়।

Q6. Lexical Scoping

একটা function কোথায় লেখা হয়েছে (physically, code-এ) তার ওপর ভিত্তি করে সে outer variable access করতে পারে কিনা তা ঠিক হয় --- call করার সময় নয়, define করার সময়।

```
function outer() {
  let name = "Rahim";
  function inner() {
    console.log(name); // outer-এর scope access করতে পারছে
  }
  inner();
}
outer();
```

Q7. Undeclared variable vs undefined

- undefined → variable declare করা হয়েছে কিন্তু কোনো value assign করা হয়নি।
- Undeclared → variable-ই declare করা হয়নি, access করলে **ReferenceError**।

```
let x;
console.log(x); // undefined

console.log(y); // ReferenceError: y is not defined
```

1.2 Functions & Closures

Q8. Closure ও Memory Leak

Closure হলো একটা function যেটা তার বাইরের (outer) scope-এর variable মনে রাখতে পারে, যদিও outer function execution শেষ হয়ে গেছে।

```
function counter() {
  let count = 0;
  return function() {
    return ++count;
  };
}
const inc = counter();
console.log(inc()); // 1
console.log(inc()); // 2
```

Memory leak কখন হয়? যখন closure একটা বড় object বা DOM element ধরে রাখে, কিন্তু সেটা আর দরকার নেই তবুও reference থাকার কারণে garbage collector সেটা clean করতে পারে না --- যেমন event listener-এ closure রেখে দিলে যেটা কখনো remove হয় না।

Q9. Function Declaration vs Function Expression

```
// Declaration - hoisted, call করার আগেও define করা যায়
function add(a, b) { return a + b; }

// Expression - hoist হয় না
const sub = function(a, b) { return a - b; };
```

Q10. this কীভাবে কাজ করে

- **Normal function:** this নির্ধারিত হয় **কে call করছে তার ওপর** (runtime binding)। Object-এর method হিসেবে call করলে সেই object, standalone call করলে undefined (strict mode) বা window।
- **Arrow function:** নিজের this থাকে না --- বাইরের (lexical) scope থেকে this নেয়।
- **Event handler:** DOM element-এ normal function দিলে this = সেই element। Arrow function দিলে this = বাইরের scope-এর (সাধারণত অকাজের)।

```
const obj = {
  name: "Karim",
  normal() { console.log(this.name); }, // Karim
  arrow: () => console.log(this.name), // undefined
};
```

Q11. bind, call, apply

তিনটাই this manually set করতে ব্যবহার হয়।

- **call(thisArg, arg1, arg2...)** → সাথে সাথে function invoke করে।
- **apply(thisArg, [argsArray])** → args array আকারে নেয়, সাথে সাথে invoke করে।
- **bind(thisArg)** → invoke করে না, নতুন একটা function **return** করে, পরে call করা যায়।

```
function greet(greeting) { console.log(greeting + " " + this.name); }
const user = { name: "Sadiah" };
```

```
greet.call(user, "Hi"); // Hi Sadia
greet.apply(user, ["Hello"]); // Hello Sadia
const bound = greet.bind(user);
bound("Hey"); // Hey Sadia
```

Use case: bind কাজে লাগে React class component-এ event handler-এর this fix করতে, বা function borrow করতে।

Q12. Currying

একটা function যেটা multiple argument না নিয়ে, একটা একটা করে argument নেয়, প্রতিবার নতুন function return করে।

```
const multiply = a => b => c => a * b * c;
console.log(multiply(2)(3)(4)); // 24
```

কেন useful? reusable, configurable function তৈরি করা যায় (partial application)।

Q13. Function Composition

ছোট ছোট function জোড়া লাগিয়ে বড় logic বানানো --- এক function-এর output অন্যটার input হয়।

```
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
const double = x => x * 2;
const addOne = x => x + 1;
const combo = compose(double, addOne); // আগে addOne, তারপর double
console.log(combo(3)); // (3+1)*2 = 8
```

1.3 Objects & Prototypes

Q14. Prototype Chaining

প্রতিটা JS object-এর একটা hidden link ([[[Prototype]]) থাকে অন্য একটা object-এর দিকে। কোনো property/method object-এ না পেলে, JS সেই link ধরে উপরের দিকে খুঁজতে থাকে, যতক্ষণ না পায় বা null-এ পৌঁছায়।

```
const animal = { eats: true };
const dog = Object.create(animal);
console.log(dog.eats); // true
```

Q15. Object.create() vs Class-based Inheritance

- Object.create(proto) → সরাসরি একটা object বানায় যার prototype তুমি নিজে সেট করে দাও। Low-level, flexible।
- class ... extends → syntactic sugar, ভেতরে ভেতরে prototype chain-ই ব্যবহার হয়, কিন্তু constructor, super(), cleaner syntax দেয়।

Q16. Property Lookup

JS একটা object-এ property খোঁজার সময় প্রথমে object নিজের মধ্যে (own property) দেখে, না পেলে prototype chain বেয়ে উপরে যায়, Object.prototype-এ গিয়ে না পেলে undefined return করে।

Q17. Shallow Copy vs Deep Copy

- **Shallow copy:** টপ-লেভেল properties কপি হয়, কিন্তু nested object/array এখনো **একই reference** শেয়ার করে।
- **Deep copy:** সব লেভেলের nested data আলাদাভাবে কপি হয়, কোনো shared reference থাকে না।

```
const original = { a: 1, nested: { b: 2 } };
const shallow = { ...original };
shallow.nested.b = 99;
console.log(original.nested.b); // 99

const deep = JSON.parse(JSON.stringify(original));
```

Q18. Object.freeze() vs Object.seal()

- freeze(): property add/delete/modify --- কোনোটাই করা যাবে না। সম্পূর্ণ immutable (শুধু top-level)।
- seal(): নতুন property add বা existing delete করা যাবে না, কিন্তু existing property-র **value change** করা যায়।

1.4 Asynchronous JavaScript

Q19. Promise States & Chaining

একটা Promise তিনটা state-এর যেকোনো একটায় থাকে: **pending** (অপেক্ষমাণ) → **fulfilled** (সফল) বা **rejected** (ব্যর্থ)। একবার fulfilled/rejected হয়ে গেলে সেটা আর বদলায় না।

```
fetchData()
  .then(res => process(res))
  .then(final => console.log(final))
  .catch(err => console.error(err));
```

প্রতিটা .then() নতুন একটা Promise return করে, তাই chain করা যায়।

Q20. async/await vs Promise (Internally)

async/await আসলে Promise-এর ওপরেই বানানো **syntactic sugar**। await মানে: এই Promise resolve না হওয়া পর্যন্ত function-এর execution ``pause" করো (কিন্তু পুরো thread block হয় না)।

```
async function getData() {
  const res = await fetch(url);
  return res.json();
}

function getData2() {
  return fetch(url).then(res => res.json());
}
```

Q21. Async function-কে await না করলে কী হয়?

Function immediately একটা **Promise** return করে, কিন্তু caller সেই result-এর জন্য অপেক্ষা করে না --- কোড আগে চলতে থাকে। যদি reject হয় সেটা **unhandled promise rejection** হয়ে যায়।

Q22. Promise.all() নিজে বানানো

```
function myPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    let results = [];
    let completed = 0;
    if (promises.length === 0) resolve([]);
    promises.forEach((p, i) => {
      Promise.resolve(p)
        .then(val => {
          results[i] = val;
          completed++;
          if (completed === promises.length) resolve(results);
        })
        .catch(reject);
    });
  });
}
```

Q23. Promise.all vs allSettled vs race vs any

Method	কখন resolve/reject করে
all	সবগুলো fulfill হলে resolve; যেকোনো একটা reject হলেই পুরোটা reject
allSettled	সবগুলো শেষ হলে resolve (fulfilled/rejected যাই হোক) --- কখনো reject হয় না
race	যেটা সবার আগে শেষ হয় (fulfilled বা rejected), সেটার result নেয়
any	যেকোনো একটা fulfilled হলেই resolve; সবগুলো reject হলে তবেই reject

Q24. Limit দিয়ে Concurrent Requests চালানো

```
async function limitedFetch(urls, limit) {
  const results = [];
  let index = 0;
  async function worker() {
    while (index < urls.length) {
      const current = index++;
      results[current] = await fetch(urls[current]).then(r => r.json());
    }
  }
  const workers = Array.from({ length: limit }, worker);
  await Promise.all(workers);
  return results;
}
```

Q25. Callback Hell ও সমাধান

একের পর এক nested callback থাকলে কোড এমন হয়ে যায় যে পড়া কঠিন, error handling জটিল --- একে ``pyramid of doom" বলে।

সমাধান: Promise ব্যবহার করে chain করা, আরও ভালো --- async/await দিয়ে synchronous-এর মতো লেখা।

1.5 Performance & Memory

Q26. Memory Leak-এর কারণ

- Global variable-এ ভুলে data রেখে দেওয়া
- Event listener remove না করা
- Closure-এ বড় object ধরে রাখা যা আর দরকার নেই
- Timer (setInterval) clear না করা
- Detached DOM nodes (JS variable-এ রেফারেন্স আছে কিন্তু DOM থেকে সরানো হয়েছে)

Q27. Garbage Collection (Mark-and-Sweep)

JS engine periodically চেক করে কোন object **reachable** (root থেকে reference chain দিয়ে পৌঁছানো যায়) --- এই object-গুলোকে **mark** করে। যেগুলো mark হয়নি (unreachable), সেগুলোকে **sweep** (delete) করে memory ফেরত নেয়।

Q28. Debounce vs Throttle

- **Debounce:** শেষ event-এর পর একটা নির্দিষ্ট সময় (delay) কেউ trigger না করলে তবেই function চালায়। যেমন search-box typing।
- **Throttle:** নির্দিষ্ট সময়ের ব্যবধানে সর্বোচ্চ একবার চালায়। যেমন scroll event handle করা।

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
function throttle(fn, limit) {
  let waiting = false;
  return (...args) => {
    if (!waiting) {
      fn(...args);
      waiting = true;
      setTimeout(() => waiting = false, limit);
    }
  };
}
```

Q29. Browser কীভাবে JS Optimize করে

- **JIT (Just-In-Time) compilation:** ঘন ঘন চলা কোড (hot code) machine code-এ compile করে ফেলে
- **Inline caching:** একই ধরনের object shape বারবার দেখলে property lookup fast করে
- **Hidden classes:** consistent object shape রাখলে V8 optimize করতে পারে
- Dead code elimination, lazy parsing ইত্যাদি

Q30. Web Worker কখন ব্যবহার করবে

যখন heavy/CPU-intensive কাজ (বড় calculation, image processing, data parsing) করতে হয় যেটা **main thread**

রক করে UI freeze করে দেবে, তখন Web Worker দিয়ে সেই কাজ আলাদা background thread-এ চালানো হয়, UI responsive থাকে।

1.6 Advanced Concepts

Q31. Immutability

একবার তৈরি হওয়ার পর data change না করা --- বদলাতে চাইলে নতুন একটি copy বানানো। এতে bug কমে, debugging সহজ হয়, React-এর মতো library-তে re-render ঠিকভাবে trigger হয়।

Q32. Pure Function

এমন function যা: (1) একই input দিলে সবসময় একই output দেয়, (2) বাইরের কোনো state modify করে না (no side effects)।

```
// pure
const add = (a, b) => a + b;
// impure - বাইরের variable modify করছে
let total = 0;
const addToTotal = x => total += x;
```

Q33. Monolithic vs Modular JS Architecture

- **Monolithic:** পুরো কোড একটাই বড় ফাইল/স্ট্রাকচারে --- ছোট প্রজেক্টে ঠিক আছে, বড় হলে maintain করা কঠিন।
- **Modular:** কোড আলাদা আলাদা module-এ ভাগ করা (ES modules import/export) --- reusable, testable, maintainable।

Q34. Event Delegation

প্রতিটা child element-এ আলাদা listener না দিয়ে, parent element-এ একটাই listener বসানো, আর event.target দিয়ে বুঝে নেওয়া কোন child click হয়েছে।

```
document.querySelector("ul").addEventListener("click", e => {
  if (e.target.tagName === "LI") console.log(e.target.textContent);
});
```

সুবিধা: memory কম লাগে, dynamically add হওয়া নতুন element-এও কাজ করে।

Q35. Symbol

Symbol একটি **unique, immutable primitive type** --- প্রতিবার Symbol() call করলে সম্পূর্ণ নতুন, আলাদা value তৈরি হয়।

Real-world use: object-এ এমন property key বানানো যেটা কারো সাথে accidentally collide করবে না, বা Symbol.iterator দিয়ে custom object-কে iterable বানানো।

```
const id = Symbol("id");
const user = { [id]: 123, name: "Rafi" };
```

Q36. Proxy

Proxy দিয়ে একটা object-এর behavior ``intercept" করা যায় --- get, set, delete ইত্যাদি operation-এ custom logic বসানো যায়।

```
const user = { name: "Nadia" };
const handler = {
  get(target, prop) {
    console.log(`accessing ${prop}`);
    return target[prop];
  }
};
const proxyUser = new Proxy(user, handler);
proxyUser.name; // logs: accessing name
```

Use case: validation, logging, reactive state (Vue 3-এর reactivity system Proxy দিয়েই বানানো)।

2 PART 2: TypeScript

2.1 Types & Interfaces

Q37. type vs interface

দুটোই object shape define করতে পারে, কিন্তু:

- interface শুধু object shape-এর জন্য, **declaration merging** সাপোর্ট করে।
- type যেকোনো কিছুর জন্য (union, intersection, primitive alias), কিন্তু merge হয় না।

```
interface User { name: string }  
interface User { age: number } // merge হয়ে গেলো: { name, age }  
  
type ID = string | number; // union - interface দিয়ে সম্ভব না
```

Q38. Union vs Intersection Types

- **Union (|)**: এই টাইপগুলোর **যেকোনো একটি** হতে পারবে।
- **Intersection (&)**: এই টাইপগুলোর **সবগুলোর property একসাথে** থাকতে হবে।

```
type A = { name: string };  
type B = { age: number };  
type Union = A | B;  
type Intersection = A & B; // name AND age দুটোই লাগবে
```

Q39. Type Inference

TypeScript নিজে থেকেই variable/return value-এর type বুঝে নেয়, explicit annotation না দিলেও।

```
let x = 5; // TS নিজেই বুঝে নেয় x: number  
function add(a: number, b: number) {  
  return a + b; // return type: number  
}
```

Q40. Optional Chaining ও Nullish Coalescing

- **?. (optional chaining)**: কোনো property null/undefined হলে error না দিয়ে সাথে সাথে undefined return করে।
- **?? (nullish coalescing)**: বাম পাশের value null বা undefined হলেই শুধু ডান পাশের fallback নেয়।

```
const city = user?.address?.city;  
const count = user.count ?? 10; // count = 0 হলেও 0-ই থাকবে
```

Q41. any vs unknown vs never

- **any:** type checking পুরোপুরি বন্ধ --- যা খুশি করা যায়, কোনো **safety** নেই।
- **unknown:** যেকোনো **value** রাখা যায়, কিন্তু ব্যবহার করার আগে **type check/narrow** করতে হবে।
- **never:** এমন টাইপ যা **কখনো ঘটে না** --- যেমন **function** যেটা সবসময় **throw** করে।

```
function fail(): never {  
  throw new Error("failed");  
}  
let val: unknown = "hello";  
if (typeof val === "string") val.toUpperCase(); // OK
```

Q42. Structural Typing

TypeScript type check করে **shape/structure** দেখে, নাম দেখে না (duck typing-এর মতো)।

```
interface Point { x: number; y: number }  
function printPoint(p: Point) { console.log(p.x, p.y) }  
const obj = { x: 1, y: 2, z: 3 };  
printPoint(obj); // কাজ করবে, shape মিলে যাচ্ছে
```

Q43. readonly

Property একবার initialize হওয়ার পর আর modify করা যাবে না --- **compile-time-এ enforce** হয়, runtime-এ কোনো **actual protection** নেই।

```
interface Config { readonly apiKey: string }  
const c: Config = { apiKey: "abc" };  
// c.apiKey = "xyz"; // Compile Error
```

2.2 Functions & Generics

Q44. Generic Constraints

Generic type-এর ওপর extends দিয়ে সীমাবদ্ধতা (constraint) দেওয়া যায়।

```
function getLength<T extends { length: number }>(item: T): number {  
  return item.length;  
}  
getLength("hello"); // OK
```

Q45. Generic Inference in Functions

Argument দেখে TypeScript নিজেই generic-এর type বুঝে নেয়।

```
function identity<T>(value: T): T { return value; }  
const result = identity("hello"); // T automatically হয় string
```

Q46. Function Overloads

একই function নাম, কিন্তু বিভিন্ন argument type/return type combination-এর জন্য আলাদা signature ঘোষণা করা।

```
function combine(a: string, b: string): string;
function combine(a: number, b: number): number;
function combine(a: any, b: any): any {
  return a + b;
}
```

Q47. void vs never

- void: function কিছু return করে না --- normally শেষ হয়।
- never: function কখনোই স্বাভাবিকভাবে শেষ হয় না (throw করে অথবা infinite loop)।

Q48. Default Generic Type

Generic-এর জন্য default type সেট করা যায়।

```
interface ApiResponse<T = any> {
  data: T;
  status: number;
}
```

2.3 Advanced Types

Q49. Mapped Types

একটা existing type-এর সবগুলো property-কে loop করে নতুন type বানানো যায়।

```
type Readonly2<T> = { readonly [K in keyof T]: T[K] };
type User = { name: string; age: number };
type ReadonlyUser = Readonly2<User>;
```

Q50. Conditional Types

Type-এর ওপর ভিত্তি করে ``if-else``-এর মতো টাইপ নির্ধারণ করা।

```
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<string>; // "yes"
type B = IsString<number>; // "no"
```

Q51. infer কীভাবে কাজ করে

Conditional type-এর ভিতরে infer দিয়ে একটা টাইপ বের করে (extract) নিয়ে নাম দেওয়া যায়।

```
type ReturnTypeOf<T> = T extends (...args: any[]) => infer R ? R : never;
function greet() { return "hi"; }
type G = ReturnTypeOf<typeof greet>; // string
```

Q52. Partial, Required, Pick, Omit, Record

- `Partial<T>` → সব property optional বানিয়ে দেয়
- `Required<T>` → সব property required বানিয়ে দেয়
- `Pick<T, K>` → নির্দিষ্ট কিছু property বেছে নিয়ে নতুন type
- `Omit<T, K>` → নির্দিষ্ট কিছু property বাদ দিয়ে নতুন type
- `Record<K, V>` → key-value shape-এর object type বানায়

```
type User = { id: number; name: string; email: string };
type PartialUser = Partial<User>;
type NameOnly = Pick<User, "name">;
type NoEmail = Omit<User, "email">;
type Scores = Record<string, number>;
```

Q53. Template Literal Types

String literal type-গুলোকে template string-এর মতো জোড়া লাগিয়ে নতুন টাইপ বানানো।

```
type Direction = "top" | "bottom";
type ClassName = `border-${Direction}`;
```

Q54. keyof ও typeof একসাথে

- `typeof` → একটা variable-এর টাইপ বের করে
- `keyof` → একটা type-এর সব key-এর union বের করে

```
const user = { id: 1, name: "Sami" };
type UserType = typeof user;
type UserKeys = keyof UserType; // "id" | "name"
```

Q55. Index Signature ও এর সীমাবদ্ধতা

Object-এ dynamic key allow করার জন্য ব্যবহার হয়।

```
interface Dict { [key: string]: number }
const scores: Dict = { math: 90, physics: 85 };
```

সীমাবদ্ধতা: সব property-কে একই টাইপ মেনে চলতে হয়, key যে সত্যিই আছে সেটা guarantee দেয় না।

2.4 Type Safety & Architecture

Q56. TypeScript কীভাবে Runtime Error ঠেকায়

Compile-time-এ ভুল টাইপের ভ্যালু pass করা, non-existent property access করা --- এসব ভুল কোড রান করার আগেই ধরিয়ে দেয়।

Q57. API Response Safely Type করা

```
interface UserResponse { id: number; name: string; email: string }

async function getUser(id: number): Promise<UserResponse> {
  const res = await fetch(`/api/users/${id}`);
  const data: unknown = await res.json();
  return data as UserResponse;
}
```

সবচেয়ে safe way: zod-এর মতো library দিয়ে runtime-এ actual structure validate করা, শুধু TS type assertion-এ ভরসা না করা।

Q58. Unknown Backend Data Handle করা

- unknown টাইপ ব্যবহার করে, তারপর typeof/in দিয়ে narrowing করা
- অথবা runtime validation library (zod, yup) দিয়ে schema validate করা
- কখনো সরাসরি any দিয়ে trust করা উচিত না

Q59. Discriminated Union

Union type-এর মধ্যে একটা common ``discriminant" property রেখে TS-কে বলে দেওয়া কোন branch-এ আছি, যাতে TS automatically narrow করতে পারে।

```
type Success = { status: "success"; data: string };
type Failure = { status: "error"; message: string };
type Result = Success | Failure;

function handle(res: Result) {
  if (res.status === "success") console.log(res.data);
  else console.log(res.message);
}
```

Q60. বড় Codebase-এ Strict Typing Enforce করা

- tsconfig.json-এ "strict": true অন করা
- noImplicitAny, noUncheckedIndexedAccess অন করা
- CI pipeline-এ tsc --noEmit check যোগ করা
- ESLint + @typescript-eslint rules ব্যবহার করা
- any ব্যবহার নিরুৎসাহিত করা (code review-এ ধরা)

2.5 TS + JS Internals

Q61. TypeScript কীভাবে JavaScript-এ Compile হয়

TypeScript compiler (tsc) সব **type annotation** স্ট্রিপ করে ফেলে এবং target অনুযায়ী plain JS বানায়।

Q62. Runtime-এ Type-এর কী হয়

কিছুই থাকে না --- types সম্পূর্ণ **erased** (মুছে ফেলা) হয়ে যায় compile করার পর।

Q63. TS DOM Typing কীভাবে Handle করে

TS-এর সাথে built-in lib.dom.d.ts ফাইল আসে যেখানে সব DOM API আগে থেকেই **typed**।

```
const input = document.querySelector("input") as HTMLInputElement;  
input.value;
```

Q64. ESM vs CommonJS (TS Projects)

- **CommonJS**: require()/module.exports, synchronous loading।
- **ESM**: import/export, static analysis সম্ভব (tree-shaking-এর জন্য ভালো)।

Q65. Types ছাড়া Third-party Library Type করা

- প্রথমে চেক করো @types/library-name আছে কিনা
- না থাকলে নিজে একটা .d.ts ফাইল বানিয়ে declare module করা

```
declare module "some-untyped-lib" {  
  export function doSomething(x: string): number;  
}
```

2.6 High-Signal System / Real-World Questions

Q66. Type-safe Form System Design

- প্রতিটা field-এর জন্য একটা schema define করা (zod/yup দিয়ে), তা থেকে TS type **infer** করা
- Generic form hook বানানো যেটা schema নিয়ে values, errors, touched state manage করে
- Validation runtime-এ, typing compile-time-এ --- schema-থেকে-infer করার কারণে দুটোই **sync** থাকবে

Q67. Frontend-Backend শেয়ার্ড Types

- **Monorepo** ব্যবহার করে একটা shared types package বানানো
- অথবা OpenAPI/GraphQL schema থেকে **auto-generate** করা types

Q68. বড় JS Project-কে TypeScript-এ Refactor করা

1. tsconfig.json লুজ সেটিংসে শুরু করা
2. ফাইল extension একটা একটা করে .js → .ts বদলানো, leaf/utility file থেকে শুরু করা
3. ধীরে ধীরে strict mode অন করা
4. Critical/shared module আগে টাইপ করা
5. CI-তে type check বাধ্যতামূলক করা

Q69. TS API-তে Backward Compatibility নিশ্চিত করা

- Breaking change এড়িয়ে নতুন জিনিস **optional** হিসেবে যোগ করা
- Deprecated fields রাখা কিন্তু @deprecated JSDoc দিয়ে চিহ্নিত করা
- Semantic versioning মেনে চলা

Q70. TypeScript-এর Trade-offs

সুবিধা: compile-time error catching, better IDE autocomplete, large codebase maintainability, self-documenting code।

অসুবিধা:

- Build step লাগে
- Learning curve --- advanced types জটিল হতে পারে
- Runtime-এ কোনো protection দেয় না, runtime validation আলাদা লাগে
- ছোট প্রজেক্টে overhead মনে হতে পারে

কীভাবে পড়বে (Study Tip)

1. প্রতিদিন একটা section ধরে code উদাহরণ নিজে হাতে লিখে try করো।
2. প্রতিটা answer নিজের ভাষায় ২-৩ লাইনে বলার practice করো।
3. Interview-এ প্রশ্ন শুনে প্রথমে concept-টা এক লাইনে বলো, তারপর ছোট example দাও।